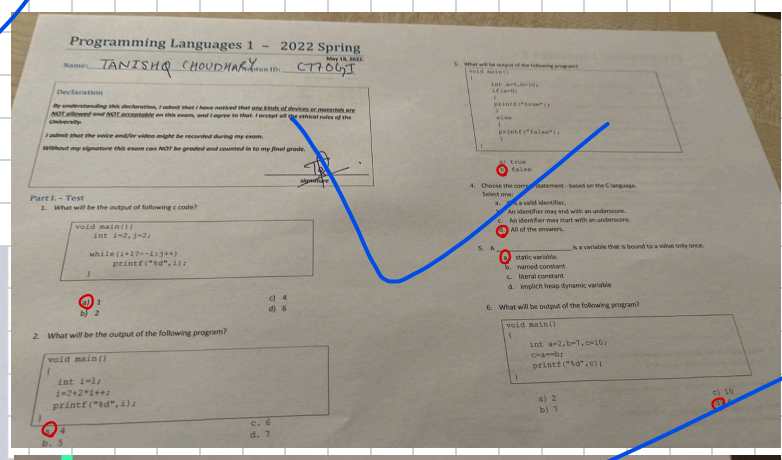
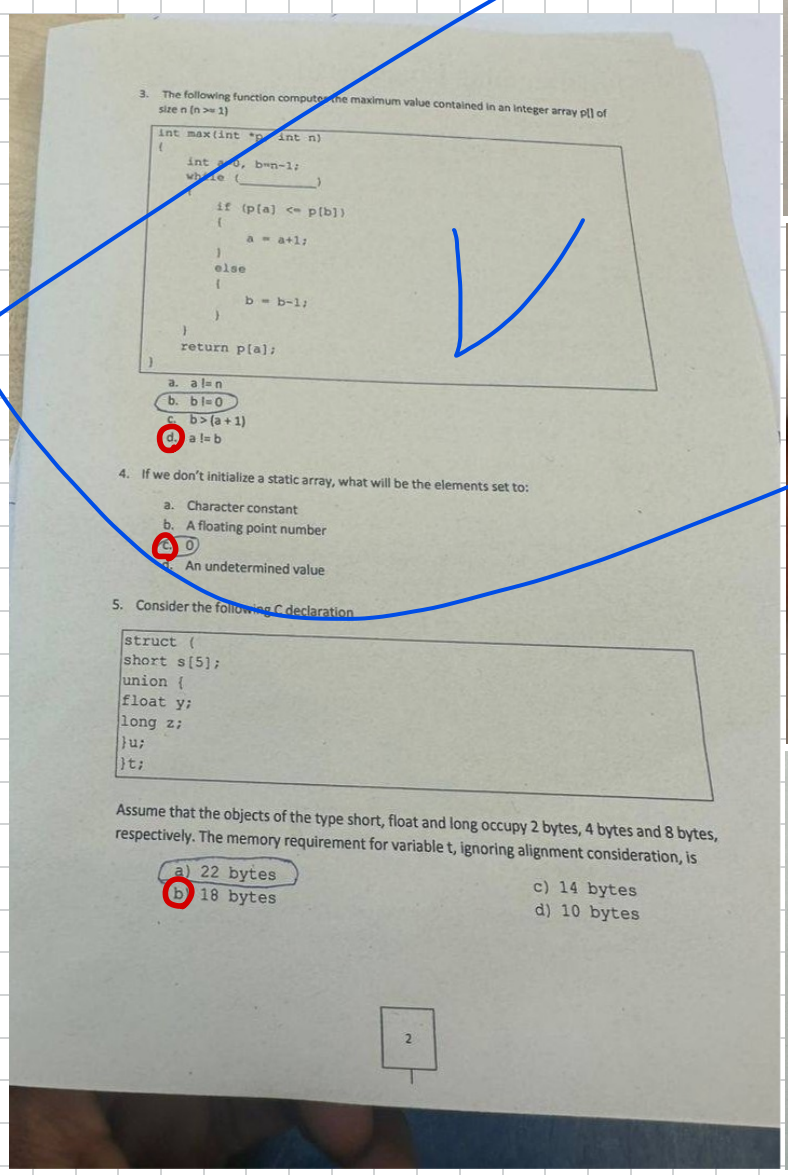
HLP1 (Ayane Takagi 2024)

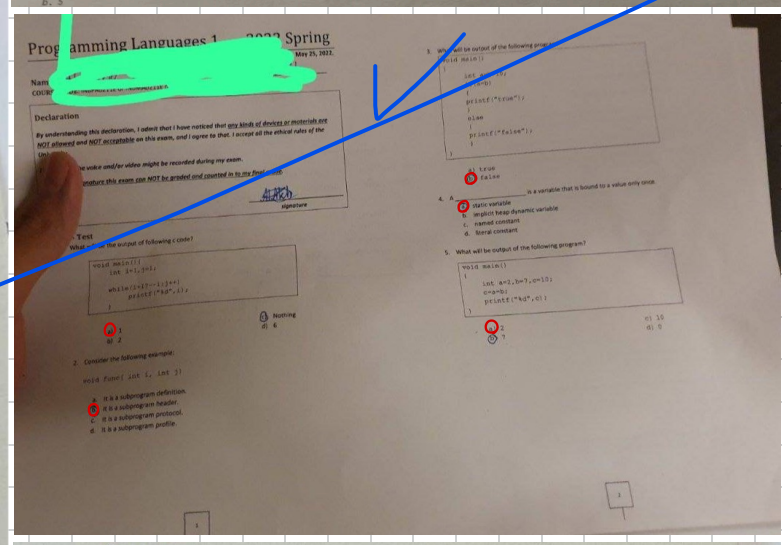
第1回目の試験日に受ける。試験内容が過去問と同じ一緒だから，スムーズにパスできる。

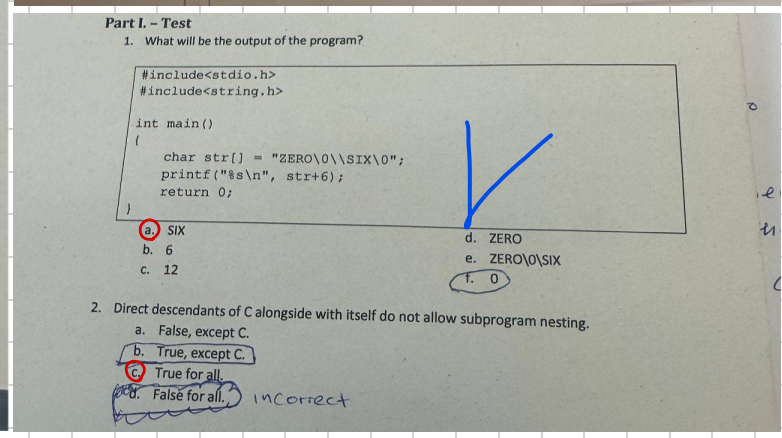
HLP1 Part 1
https://docs.google.com/forms/d/e/1FAIpQLSc_m72JoDCNTBH0C-A_kv8XRS6WKE9xjKAfhk5qRixQ14nvhw/viewform?embedded=true

Textbook
https://www.cs.csubak.edu/~jyang/Robert%20W.%20Sebesta%20-%20Concepts%20of%20Programming%20Languages-Pearson%20(2015).pdf


Programming Languages 1 – 2022 Spring
May 18, 2022.
Name: TANISHQ CHOUDHARY Neptun ID: CT7GJ1
Declaration
By understanding this declaration, I admit that I have noticed that any kinds of devices or materials are NOT allowed and NOT acceptable on this exam, and I agree to that. I accept all the ethical rules of the University.
I admit that the voice and/or video might be recorded during my exam.
Without my signature this exam can NOT be graded and counted in to my final grade.
signature
Part I. – Test
1. What will be the output of following c code?
void main(){ int i=2,j=2; while(i+1?--i:j++) printf("%d",i); }
a) 1 b) 2 c) 4 d) 6
2. What will be the output of the following program?
void main() { int i=1; i=2+2*i++; printf("%d",i); }
a) 4 b. 5 c. 6 d. 7
3. What will be output of the following program?
void main() { int a=0,b=10; if(a==0) printf("true"); else printf("false"); }
a) true b) false
4. Choose the correct statement - based on the C language.
Select one:
a. Not a valid identifier.
b. An identifier may end with an underscore.
c. An identifier may start with an underscore.
d. All of the answers.
5. A _____ is a variable that is bound to a value only once.
a. static variable.
b. named constant.
c. literal constant.
d. implicit heap dynamic variable.
6. What will be output of the following program?
void main() { int a=2,b=7,c=10; c=a==b; printf("%d",c); }
a) 2 b) 7 c) 10 d) 0


3. The following function computes the maximum value contained in an integer array p[] of size n (n >= 1)
int max(int *p, int n)
{
int a=0, b=n-1;
while (_____)
{
if (p[a] <= p[b])
{
a = a+1;
}
else
{
b = b-1;
}
}
return p[a];
}
a. a != n
b. b != 0
c. b > (a + 1)
d. a != b
4. If we don't initialize a static array, what will be the elements set to:
a. Character constant
b. A floating point number
c. 0
d. An undetermined value
5. Consider the following C declaration
struct {
short s[5];
union {
float y;
long z;
}u;
}t;
Assume that the objects of the type short, float and long occupy 2 bytes, 4 bytes and 8 bytes, respectively. The memory requirement for variable t, ignoring alignment consideration, is
a) 22 bytes
b) 18 bytes
c) 14 bytes
d) 10 bytes
2


Programming Languages 1 – 2022 Spring
Declaration
Part I. – Test
1. What will be the output of following c code?
2. Consider the following example:
void Func(int i, int j)
a. It is a subprogram definition.
b. It is a subprogram header.
c. It is a subprogram protocol.
d. It is a subprogram profile.
3. What will be output of the following program?
4. A _____ is a variable that is bound to a value only once.
a. static variable
b. implicit heap dynamic variable
c. named constant
d. literal constant.
5. What will be output of the following program?


Part I. – Test
1. What will be the output of the program?
#include<stdio.h>
#include<string.h>
int main()
{
char str[] = "ZERO\0\\SIX\0";
printf("%s\n", str+6);
return 0;
}
a. SIX
b. 6
c. 12
d. ZERO
e. ZERO\0\SIX
f. 0
2. Direct descendants of C alongside with itself do not allow subprogram nesting.
a. False, except C.
b. True, except C.
c. True for all.
d. False for all.
Incorrect

Void main () {
Int i =2 , j = 2;
while(i + 1? –i : j++)
printf((“%d”, i);
}
a) 1 b) 2 c) 4 d) 6
void main () {
int i = 1;
i = 2+2*i++;
printf(“%d”,i);
}
a) 4 b) 5 c) 6 d) 7
Void main () {
Int a = 0 , b = 10;
If(a=0)
{
Printf(“true”);
}
else
{
Printf(“false”);
}
}
a) true b) false
#include <stdio.h>
void main () {
int a=100;
if(a>10)
printf(“M.S Dhoni”);
else if(a>20)
printf(“M.E.X Hussey”);
else if(a>30)
printf(“A.B de villiers”);
}
1 . M.S Dhoni
3 . M.S Dhoni
M.E.X Hussey
A.B de villiers
2 . A.B de villiers
4 . compilation error.More than one conditions are true
5 . None of the above
Void main () {
Int a=2 , b=7 , c=10;
C=a==b;
Printf(“%d”,c);
}
a . 2 b . 7 c . 10 d . 0

3. In the following program fragment, a <= b will be printed if?
if(a > b)
printf("a > b");
else
printf("else part");
printf("a <= b");
a. a < b
b. a > b
c. does not depend upon if condition
d. a == b
4. In the below statement, ptr1 and ptr2 are uninitialized pointers to int i.e. they are pointing to some random address that may or may not be valid address.
int* ptr1, ptr2;
a. False, because they are pointing to the null address.
b. True, because they are in the same statement.
c. True, because they are not initialized.
d. False, because they are different type variables.
5. What will be output of the following program?
void main()
{
int a=7,b=2,c=10;
c=a=b;
printf("%d",c);
}
a) 2
b) 10
c) 7
d) 0

Programming Languages 1 – 2022
December 20, 2022.
Name: _____ Neptun ID: _____
COURSE CODE: INBPA0211E or INBMA0211E or INGK301 or INJK301 or INHK301
Declaration
By understanding this declaration, I admit that I have noticed that any kinds of devices or materials are NOT allowed and NOT acceptable on this exam, and I agree to that. I accept all the ethical rules of the University.
I admit that the voice and/or video might be recorded during my exam.
Without my signature this exam can NOT be graded and counted in to my final grade.
signature
Part I. – Test
1. In the following program fragment, s2 will be executed if?
if(a > b)
if(b > c)
s1;
else
s2;
a. a > b and b <= c
b. a <= b
c. b <= c and a <= b
d. b > c
2. Direct descendants of C alongside with itself do not allow subprogram nesting.
a. False, except C.
b. True, except C.
c. True for all.
d. False for all.

Programming Languages 1

Part II. – Questions

1) Which produces faster program execution: a compiler or a pure interpreter? (1 point)
[Explain your answer!] (3 points)

2) Define the meaning of syntax and semantics! (2 points)

3) How can you describe a variable? What is a variable? (1 point)
List and explain all the attributes of a variable (excl. lifetime and scope)! (4 points)

4) What is the definition of block? (2 points)

5) What does it mean when a variable is static? (4 points)
(Explain its advantages and disadvantage!)

6) Define scope and lifetime! (2 points)
Explain C's scoping and lifetime in the following code fragment: (3 points)

```
void sub(){
   int count;
   …
  while( … ){
   int count;
   count ++
   …
  }
  …
}
void printheader(){
 int sum;
 …
 sub();
}
```

scope : within the while block
lifetime : while the while block is being executed.

scope : within the printheader function
life-time : while the printheader function is called.

7) What are the design issues for multiple-selection statements? (2 points)
[Explain your answers for C especially!] (3 points)

Programming Languages 1

Part II. – Questions

1) Define the concept of type binding for static and dynamic cases! (4 point)
[Explain your answer with examples for each one! (expl./impl.; type inf.; compil,intrpt)]

2) Define steps to the semantics of a call to a "simple" subprogram! (2 points)
Show an example!

3) How can you describe a variable? What is a variable? (1 point)
List and explain all the attributes of a variable (incl. lifetime and scope)! (6 points)

4) What is a record as a data type? (1 points)
Compare it with arrays! (2 points)

5) What does it mean when a variable is stack-dynamic? (3 points)
(Explain its advantages and disadvantage!)

6) Define the two fundamental kinds of subprograms! (2 points)
What are the similarities and differences between them? (3 points)

7) What is an associative array? (2 points)
[Explain your answers for a chosen language!] (2 points)

8) What could be a **problem** with nested selection statement in C based languages? (2 points)

9) What are special words? (1 points)
Explain your answer with its subtypes and examples! (3 points)

10) Explain the Pass-by-result model in details with examples! (4 points)
Discuss the well-known potential problem with it!

Programming Languages 1

Part II. – Questions

1) Define the concept of *type binding* for *static* and *dynamic* cases! (4 point)
[Explain your answer with examples for each one! (expl./impl.; type inf.; compil,intrpt)]

2) Define steps to the semantics of a call to a "simple" subprogram! (2 points)
Show an example!

3) How can you describe a variable? What is a variable? (1 point)
List and explain all the attributes of a variable (incl. lifetime and scope)! (6 points)

4) What is an array as a data type? (1 points)
Compare it with records! (2 points)

5) What does it mean when a variable is heap-dynamic? (3 points)
(Explain its advantages and disadvantage!)

6) Define the two fundamental kinds of subprograms! (2 points)
What are the similarities and differences between them? (3 points)

7) What is a union type? (2 points)
[Explain your answers for a chosen language!] (2 points)

8) What could be a **problem** with nested selection statement in C based languages? (2 points)

9) What is a reserved word? (1 points)
Explain your answer with examples and indicating the difference from keywords! (3 points)

10) Explain the Pass-by-reference model in details with examples! (4 points)
Discuss the well-known potential problem with it!

Programming Languages 1 Part II. - Questions

1) Explain the three reasons why lexical analysis is separated from syntax analysis. (4 point)
[Explain your answer with examples for each one]

2) What is a reserved word? (1 points)
3) What is a static ancestor of a subprogram?
What is a dynamic ancestor of a subprogram? (2 points)

4) What are the advantages and disadvantages of dynamic type binding? (2 points)
5) What is the referencing environment of a statement? (2 points)

6) What is an overloaded operator? (2 points)

7) What does it mean when a variable is stack dynamic?
(Explain its advantages and disadvantage) (3 points)

8) Define scope and lifetime! (2 points)
Explain C's scoping and lifetime for all sum variables in the code Fragment: (3 points)

```
void sub () {
int sum;
…
while(...) {
 int sum;
sum ++
…
  }
…
}
void printheader () {
int sum;
…
sub () ;
}
```

9) What is the Pass-by-Result model of parameter passing? (6 points)
Explain in your answers the parameter collision problem!

10) How does short-circuit expression evaluation works? (2 points)

11) What is a generic subprogram? (2 points)

12) What is a coroutine? (2 points)

13) In what way is C's for statement more flexible than that of many other languages? [Show examples:] (3 points)

Programming Languages 1

Part II. – Questions

1) Define static binding and dynamic binding! (4 point)
[Explain your answer with examples for each one!]

2) Define the subprogram protocol! (2 points)

3) How can you describe a variable? What is a variable? (1 point)
List and explain all the attributes of a variable (excl. lifetime and scope)! (4 points)

4) Define the referencing environment for a static-scoped language! (2 points)

5) What does it mean when a variable is implicit heap dynamic? (3 points)
(Explain its advantages and disadvantage!)

6) Define scope and lifetime! (2 points)
Explain C's scoping and lifetime **for all count variables** in the code fragment: (3 points)

```
void sub(){
  int count;
  …
 while( … ){
  int count;
  count ++
  …
 }
 …
}
void printheader(){
 int sum;
 …
 sub();
}
```

7) What are the design issues for arrays? (2 points)
[Explain your answers for C!] (3 points)

8) What is **unusual** about C's multiple selection statement? (2 points)

9) What are the two common problems with pointers? (2 points)
Explain your answer! (2 points)

10) What are the three semantic models of parameter passing? (4 points)
Explain the Pass-by-value-result model in details!

Question

1) Types of binding:

➢ Static binding:

➢ Dynamic binding:

Answer

1) Types of binding:

➢ Static binding: a binding is static if it first occurs before run time and remains unchanged throughout program execution.

➢ Dynamic binding: a binding is dynamic if it first occurs during run time or can change during program execution.

Question

2) Variable declarations:

➢ Explicit declaration:

➢ Implicit declaration:

Answer

2) Variable declarations:

➢ Explicit declaration: is a program statement that lists variable names and specifies that they are a particular type.

example in c:

int x;

float pi = 3.14;

char letter = 'A';

➢ Implicit declaration: is a means of associating variables with types through default conventions, rather than declaration statements.

example in python:

x = 10;

pi = 3.14;

letter = 'A':

Question

3) Type inference?

Answer

46) Type inference?

is implicit type declarations using context which is the type of the value assigned to the variable in a declaration statement.

example in c#:

var sum = 0; (type; int)

var total = 0.0; (type; float)

var name = "Fred"; (type; string)

Question

4) Implementation Methods:

❖ Compiler:

❖ Pure interpretation:

❖ Hybrid Implementation System:

Answer

4) Implementation Methods:

❖ Compiler: Programs are translated into machine code.

❖ Pure interpretation: The program is translated into machine code for a particular CPU one instruction at a time and executes immediately.

❖ Hybrid Implementation System: A compromise between compiler and pure interpretation.

Question

5) What is Binding and Binding Time?

➢ Binding:

➢ Binding Time:

Answer

5) What is Binding and Binding Time?

➢ Binding: is an association between an attribute and an entity
(Such as between a variable and its type).

➢ Binding Time: is the moment in the program's life cycle when this association occurs.

Question

6) What is a variable? [list and explain all the attributes of a variable]

➢ Name:

➢ Address:

➢ Type:

➢ Value:

➢ Lifetime:

➢ Scope:

Answer

6) What is a variable? [list and explain all the attributes of a variable]

A variable is an abstraction of a computer memory cell or collection of cells.

➢ Name: is a string of characters used to identify some entity in a program.

➢ Address: is the machine memory address with which it is associated.

➢ Value: is the contents of the memory cell or cells associated with the variable.

➢ Type: Is the range of values that a variable can store.

Question

7) What does it mean when a variable is static?

Advantage:

Disadvantage:

Answer

7) What does it mean when a variable is static?

Static variables are bound to memory cells before execution begins and remain bound to the same memory cells until execution terminates.

Advantage: Efficiency.

Disadvantage: Reduced flexibility.

Question

8) What does it mean when a variable is (Explicit) Heap Dynamic?

Advantage:

Disadvantage:

Answer

8) What does it mean when a variable is (Explicit) Heap Dynamic?

Are nameless memory cells that are allocated and deallocated by explicit run-time instructions written by the programmer.

Advantage: Providing Dynamic storage management.

Disadvantage: Inefficient and Unreliable.

Question

9) what does it mean when a variable is Implicit Heap Dynamic?

Advantage:

Disadvantage:

Answer

9) what does it mean when a variable is Implicit Heap Dynamic?

are bound to heap storage only when the variable is assigned a
value.

Advantage: Flexibility allowing generic code to be written.
Disadvantage: The loss of error detection by the compiler.

Question

10) What does it mean when a variable is stack-dynamic?

Advantage:

Disadvantages:

Answer

10) What does it mean when a variable is stack-dynamic?

are those whose storage bindings are created when their declaration statements are elaborated, but whose types are statically bound.

Advantage: allows recursion; conserves storage.
Disadvantages: Overhead of allocation and deallocation
possibly slower accesses (indirect addressing)
Subprograms cannot be history-sensitive

Question

11) Parameter passing methods:

➢ Pass-by-value: (In-Mode)
➢ Pass-by-result: (Out-Mode)
➢ Pass-by-value-result: (In-Out-Mode)
➢ Pass-by-reference: (In-Out-Mode)
➢ Pass-by-name: (In-Out-Mode)

Answer

11) Parameter passing methods:

➢ Pass-by-value: (In-Mode)
❖ The value of the actual parameter is used to initialize the corresponding formal parameter.

➢ Pass-by-result: (Out-Mode)
❖ Just before control is transferred back to the caller,
the value of the formal parameter is transmitted back to the actual parameter.

➢ Pass-by-value-result: (In-Out-Mode)
❖ Is a combination of Pass-by-value and Pass-by-result.

➢ Pass-by-reference: (In-Out-Mode)
❖ Passing the memory address of where a variable is stored.

➢ Pass-by-name: (In-Out-Mode)
❖ Is one where the variable name is passed.

Question

12) Record? Array?

Answer

12) Record? Array?

➢ Record: Is a collection of data elements
that the individual elements are identified by names.

➢ Array: A series of memory locations each of which holds a single item of data.

Question

13) Associative Array?

Answer

13) Associative Array?

is an unordered collection of data elements that are indexed by an equal number of values called key.

Question

14) Which procedure executes faster, a compiler or pure interpreter? Explain.

Answer

14) Which procedure executes faster, a compiler or pure interpreter? Explain.

A compiler is faster because
a pure interpreter reads the source code line by line and executes it as it goes
while the compiler takes the source code and converts it into machine language.

Question

15) Define the meaning of syntax and semantics.

Answer

15) Define the meaning of syntax and semantics.

➢ Syntax: The structure of the expression, statement, and program unit.
➢ Semantics: The meaning of the expression, statement, and program unit.
➢ Syntax and Semantics: provide a language definition.

Question

16) What is the definition of block?

Answer

16) What is the definition of block?

is a compound statement that lets you group any number of data declarations
into one statement using curly brackets {}.

Question

17) what are the design issues for multiple-selection statements?

(Explain your answers for C especially!)

Answer

17) what are the design issues for multiple-selection statements?

(Explain your answers for C especially!)

➢ What is the form and type of the expression that controls the section?

➢ How are the selectable segments specified?

➢ Is execution flow through the structure restricted to include just a single selectable

segment?

➢ How are the case values specified?

➢ How should unrepresented selector expression values be handled, if at all?

Question

18) How does C support Relation and Boolean expressions?

Answer

18) How does C support Relation and Boolean expressions?

C supports Relation expressions by using signs such as >, >=, <, <=, ==.

And supports Boolean expressions by using 1 and 0 to represent true and false

where 1 is true and 0 is false.

Question

19) What are the two common problems with pointers?

Answer

19) What are the two common problems with pointers?

➢ Dangling pointer: When an object with an incoming reference is deleted or deallocated

without modifying the value of the pointer,

leaving the pointer still pointing to the memory location of the deallocated memory.

➢ Lost Heap-Dynamic Variable: An allocated Heap-Dynamic Variable

that is no longer accessible to the user program

but still contains some useful data (Often called “garbage”).

Question

20) In what way is C’s “for” statement more flexible than that of other languages?

Answer

20) In what way is C’s “for” statement more flexible than that of other languages?

➢ All 3 expressions are optional.

➢ Each expression can be an entire statements or sequence.

➢ Expression value is the value of the last statement.

➢ Everything can be changed within the loop.

➢ Legal to branch into the body of the loop.

Question

21) Union?

Answer

21) Union?

is a type whose variables are allowed to store different type values
at different times during execution.

Question

22) What is coroutine?

Answer

22) What is coroutine?

It's a computer program component that generalizes subroutines for non-preemptive multitasking, by allowing execution to be suspended and resumed.

Question

23) Define the referencing environment.

Answer

23) Define the referencing environment.

Referencing environment: the collection of all the variables visible in the statement.

Static-scoped language: is the local variable, plus all visible variables in all enclosing scopes.

Dynamic-scoped language: is the local variable, plus all visible variables in all active subprograms.

Question

24) How does short-circuit expression evaluation work?

Answer

24) How does short-circuit expression evaluation work?

A short circuit expression is one
in which the result is determined without evaluating the whole expression.
EXAMPLE (a*b)*(12+b) a = 0, b = 2.

Question

25) How is a lexical analyzer a part of a syntax analyzer?

Answer

19) How is a lexical analyzer a part of a syntax analyzer?

A lexical analyzer server as a front-end to the syntax analyzer.

➢ Lexical analyzer: acts as an interface
between the source code and the rest of the phases of a compiler.

➢ Syntax analyzer: Takes the input from the lexical analyzer as a token stream.

Question

26) What are the differences between lexical and syntax analysis?

= [Explain the three reasons why lexical analysis is separated from syntax analysis.
(Explain your answer with examples for each one)]

Answer

26) What are the differences between lexical and syntax analysis?

➢ Simplicity: Techniques for lexical analysis are less complex
than those required for syntax analysis.

➢ Efficiency: Syntax analyzer is more efficient
as lexical analyzer requires a significant portion of total completion time

➢ Lexical analyzer is platform dependent; Syntax analyzer is platform independent

➢ Lexical analyzer is the first phase of compilation; Syntax analyzer is the second phase of compilation.

Question

27) What is the generic subprogram and overloaded subprogram?

Answer

27) What is the generic subprogram and overloaded subprogram?

➢ Generic subprogram: is one whose computation can be done
on data of different types and calls.
➢ Overloaded subprogram: has the same name as another subprogram
in the same referencing environment.

Question

28) What is a reserved word?

=[What is a reserved word?
Explain your answer with examples and indicate the differences from keywords!]

Answer

28) What is a reserved word?
=[What is a reserved word?
Explain your answer with examples and indicate the differences from keywords!]

➢ Reserved word: a special word in a programming language
that has a fixed meaning and therefore cannot be redefined.

➢ Keywords: are a subset of reserved words
with specific meanings within the language.

Question

29) What is operator overload?

Answer

29) What is operator overload?

It's a potential problem that a single operator symbol has more than one meaning.

Question

30) Define Special words.

= [What are special words?
Explain your answer with its subtypes and examples!]

Answer

30) Define Special words.

= [What are special words?
Explain your answer with its subtypes and examples!]

Special words are used to separate the syntactic part of statements and programs.
Subtypes: keywords, reserved words
Ex: Fortran

Question

31) What is a nested subprogram?

= [What could be a problem with nested selection statements in C-based languages?]

Answer

31) What is a nested subprogram?

= [What could be a problem with nested selection statements in C-based languages?]

➢ Nested subprogram: is a function which is defined within another function.

➢ Problem: If the selection statement is nested in "then",
it is not clear which "else" is associated with.
A solution to this is to put an inner if compound.

```
IF (num == 1)
{
        If (result == 1)
                return 1;
}
ELSE return 0;
```

Question

32) Subprogram types:

= [Define the two fundamental kinds of subprograms!
What are the similarities and differences between them?]

Answer

32) Subprogram types:

= [Define the two fundamental kinds of subprograms!
What are the similarities and differences between them?]

➢ Function: A block of code that only runs when it is called.
➢ Procedure: A collection of statements that define parameterized computations

➢ Similarities: names can be called /accept input parameters.

➢ Differences:
Function: have return value; are used in expressions or assigned to variables.
Procedure: no return value; are invoked for their execution;

Question

33) Define the subprogram protocol.

Answer

33) Define the subprogram protocol.

Is the parameter profile plus if it's a function its return type.

Question

34) what are the design issues for arrays? —

Answer

34) what are the design issues for arrays?

➢ What type is legal for subscripts?

➢ When is subscript range bound?

➢ When does array allocation take place?

➢ Are ragged or triangular multi-dimensional arrays allowed?

➢ Can arrays be initialized when they have their storage allocated?

➢ What kind of slices are allowed?

Question

35) What is unusual about C's multiple selection statement?

Answer

35) What is unusual about C's multiple selection statement?

The C switch statement has virtually no restrictions on the placement of the case expressions,
which are treated as if they were normal statement labels.
This laxness can result in a highly complex structure within the switch body.
The reliability issue with the design arises
when the mistaken absence of a break statement.

(C の複数選択ステートメントの何が異常?
C の switch statement に、case expression の配置に関して事実上制限がないこと。
これらは通常のステートメント ラベルのように扱われてしまう。
この緩みにより、スイッチ本体内の構造が非常に複雑になる可能性が！
設計の信頼性の問題は、break ステートメントが誤って存在しない場合に発生。)

Question

36) what are the three semantic models of the parameter passing?

Answer

36) what are the three semantic models of the parameter passing?

➢ In-mode: Formal parameter can receive data from actual parameter.

➢ Out-mode: Formal parameter can transmit data to the actual parameter.

➢ In-Out-mode: Can do both.

Question

37) Define Static scoping and Dynamic scoping.

= [Define static scope! Show an example of nested scope in the C language!]

Answer

37) Define Static scoping and Dynamic scoping.

= [Define static scope! Show an example of nested scope in the C language!]

➢ Static scoping: The scope of the variable can be determined statically (before execution).

➢ Dynamic scoping: The scope of the variable can be determined dynamically (during run-time).

➢ Ex of nested scope:

```
#include <stdio.h>
int main()
{
        int x = 5;  // Outer scope variable
        {
                int y = 10;  // Inner scope variable
        }
                // y is not accessible here, it was only in the inner block
        return 0;
}
```

Question

38) Array categories:

➢ Static array:

➢ Fixed stack-dynamic array:

➢ Stack-dynamic array:

➢ Fixed heap-dynamic array:

➢ Heap-dynamic array:

Answer

38) Array categories:

➢ Static array: subscript range is statically bound
and storage allocation is static (done before run-time).

➢ Fixed stack-dynamic array: subscript range is statically bound
but the allocation is done during execution.

➢ Stack-dynamic array: subscript range is dynamically bound,
and the storage allocation is dynamic (during execution).

➢ Fixed heap-dynamic array: subscript range is dynamically bound,
and the storage allocation is dynamic,
but they are both fixed after the storage is allocated.

➢ Heap-dynamic array: subscript range is dynamically bound,
and the storage allocation is dynamic,
and can change any number of times during the array's lifetime.

Question

39) Define steps to the semantics of a call to a "simple" subprogram!
Show an example!

Answer

39) Define steps to the semantics of a call to a "simple" subprogram!
Show an example!

1. Save the execution status of the current program unit.
2. Compute and pass the parameters.
3. Pass the return address to the caller.
4. Transfer control to the called.

```
int addNum(int a, b)
{
        int result = a + b;
        return result;
}
int main()
{
        int x = 5;
        int y = 10;
        int sum = addNum(x, y);
        printf("Sum = %d", sum);

        return 0;
}
```

Question

40) What are the advantages and disadvantages of dynamic-type binding?

Advantage:

Disadvantages:

Answer

40) What are the advantages and disadvantages of dynamic-type binding?

Advantage: Flexibility

Disadvantages: High cost and difficulty of type error detection by the compiler